DEEP EXPLANATION NOTES | DIAGRAMS | LIVE EXAMPLES

# Java Web Technology

## Servlet | JSP | MVC | EJB | Session

- MVC + Architecture
- JavaBean Theory
- EJB & Session Beans
- Servlet vs JSP Session
- Servlet & JSP Life Cycle
- 9 Implicit Objects
- JSP Elements / Scriptlet
- Interview Q&A

Start: Open Index >

Tip: Click the cards on the Index page. Every page has a "Back to Index" link.
You can also jump to chapters from the Bookmarks panel of your PDF reader.

# INDEX

**Tap / click any card** to jump straight to that chapter. Every page has a **Back to Index** link at the bottom, and you can also use the **Bookmarks** panel of your PDF reader.


1 **MVC Pattern + Live Example**
Model-View-Controller, flow, Student Result app

2 **Architecture (Java EE / Web)**
Tiers, web container, online banking example

3 **Beans Theory (JavaBean)**
Rules, properties, useBean, JavaBean vs EJB

4 **EJB - Enterprise JavaBeans**
Container, types, services, code

5 **Session & Session Tracking**
Stateless HTTP, cookies, HttpSession API

6 **Session Beans (EJB)**
Stateless, Stateful, Singleton + lifecycle

7 **Servlet Session vs JSP Session**
Are they different? Full comparison

8 **JSP vs Servlet**
13-point difference + code comparison

9 **Servlet & JSP Life Cycle**
init/service/destroy, jspInit/_jspService

10 **What is JSP & Why JSP**
Definition, need, advantages, anatomy

11 **Which is Faster: JSP or Servlet?**
The truth, first-request cost, tuning

12 **9 Implicit Objects of JSP**
request, response, out ... exception

13 **JSP Elements & Scriptlet**
Scriptlet, expression, declaration, directive

14 **Revision, Q&A, Cheat Sheet**
Interview questions + mnemonics


## How to read this PDF (colour guide)

**NOTE**
Blue box = extra information / something to remember.

**TIP**
Green box = best practice / shortcut.

**WARNING**
Orange box = common mistake or danger zone.

**EXAM / INTERVIEW POINT**
Purple box = a line that is directly asked in exams or interviews.

**Note:** Code uses `javax.servlet.*` (classic syllabus / Tomcat 9 style). In Tomcat 10+ / Jakarta EE 9+ the package is `jakarta.servlet.*` - only the package name changed, the concepts are identical.

# 1 MVC Pattern + Live Example

Model - View - Controller

## 1.1 What is MVC?

**MVC (Model-View-Controller)** is a **design pattern** that splits an application into 3 separate parts so that each part has a clear job and changing one part does not break the others. This idea is called **Separation of Concerns**.

| MODEL (data + logic) | VIEW (UI) | CONTROLLER (manager) |
|---|---|---|
| **Job:** fetch/save data from the DB, apply business rules (e.g. marks to grade).<br>**In Java Web:** JavaBean, POJO, DAO, Service, EJB.<br>**Does not know:** how the data will look on screen. | **Job:** show the Model's data to the user.<br>**In Java Web:** JSP, HTML, EL, JSTL.<br>**Rule:** no DB code / heavy logic here. | **Job:** receive the request, validate input, call the Model, choose the right View.<br>**In Java Web:** Servlet (or Filter).<br>**Rule:** do not write HTML here. |

> **NOTE**
> **Restaurant analogy:** Customer = Browser, Waiter = Controller (takes the order, tells the kitchen), Kitchen = Model (actually prepares the food), Plating = View (how the food is served). The waiter does not cook and the kitchen never talks to the customer.

## 1.2 MVC Architecture Diagram

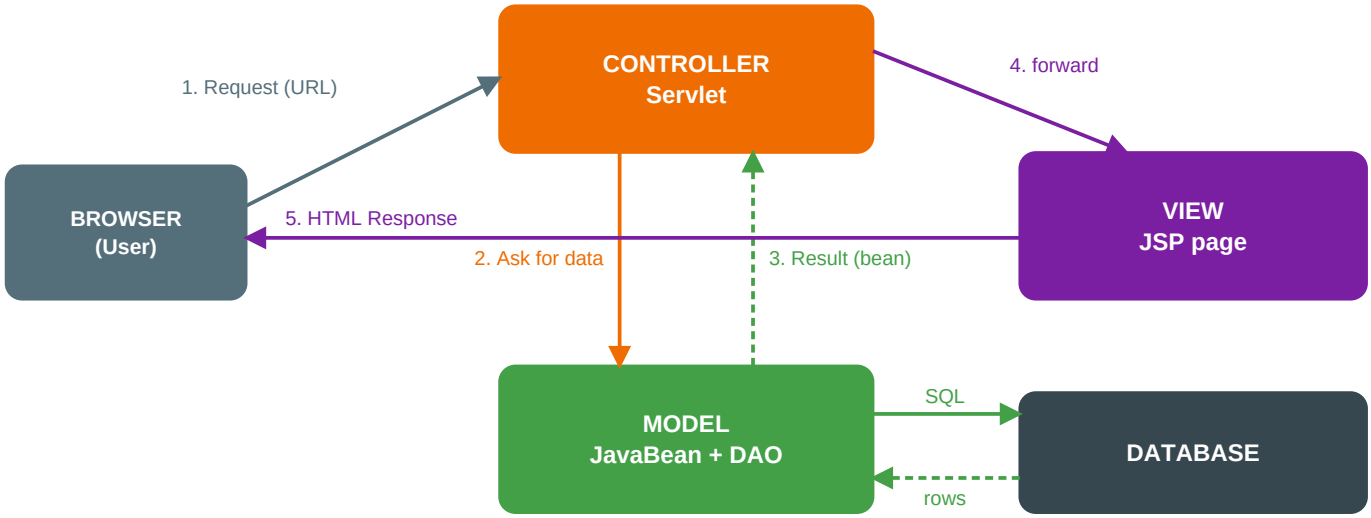



*Fig 1.1 - Request flow of MVC (Model 2) in Java Web*

### Flow step by step

- **1.** The user types a URL / submits a form in the browser, and the request reaches the **Controller (Servlet)**.
- **2.** The Controller reads the parameters (`request.getParameter`) and gives the work to the Model (DAO/Bean).
- **3.** The Model talks to the DB, fills a **bean** with data and returns it to the Controller.
- **4.** The Controller stores the bean with `request.setAttribute()` and `forward()`s to the **View (JSP)**.
- **5.** The JSP puts the bean data into HTML and builds the **response**.
- **6.** The HTML is displayed in the user's browser.

## 1.3 Live Example: Student Result Portal

**Scenario:** A student enters a roll number and sees the result (name, marks, grade). Each file below is one MVC part.

**Project**

```
StudentApp/
 |-- src/com/app/model/Student.java            -> MODEL  (JavaBean, data holder + grade rule)
 |-- src/com/app/dao/StudentDAO.java           -> MODEL  (DB access using JDBC)
 |-- src/com/app/controller/ResultServlet.java -> CONTROLLER (handles request, forwards)
 |-- WebContent/index.html                     -> input form
 |-- WebContent/result.jsp                     -> VIEW   (display only)
```

## Step 1: index.html (input form)

**HTML**

```
<form action="result" method="post">
   Roll No: <input type="text" name="roll">
   <input type="submit" value="Show Result">
</form>
```

## Step 2: Student.java (Model - JavaBean)

**Java**

```
package com.app.model;
import java.io.Serializable;

public class Student implements Serializable {
    private int roll;
    private String name;
    private int marks;

    public Student() { }                                    // no-arg constructor (bean rule)
    public int getRoll()           { return roll; }
    public void setRoll(int roll)  { this.roll = roll; }
    public String getName()        { return name; }
    public void setName(String n)  { this.name = n; }
    public int getMarks()          { return marks; }
    public void setMarks(int m)    { this.marks = m; }
    public String getGrade()       { return marks >= 75 ? "A" : marks >= 50 ? "B" : "C"; }  // business rule
}
```

## Step 3: StudentDAO.java (Model - DB logic)

**Java**

```
public class StudentDAO {
    public Student findByRoll(int roll) throws Exception {
        String sql = "SELECT name, marks FROM student WHERE roll = ?";
        try (Connection con = DBUtil.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, roll);
            ResultSet rs = ps.executeQuery();
            if (rs.next()) {
                Student s = new Student();
                s.setRoll(roll);
                s.setName(rs.getString("name"));
                s.setMarks(rs.getInt("marks"));
                return s;                          // return the filled bean
            }
        }
        return null;                               // roll number not found
    }
}
```

## Step 4: ResultServlet.java (Controller)

```java
@WebServlet("/result")
public class ResultServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        int roll = Integer.parseInt(req.getParameter("roll"));     // 1. read input
        try {
            Student s = new StudentDAO().findByRoll(roll);          // 2. get data from Model
            req.setAttribute("student", s);                         // 3. keep it in request
            req.getRequestDispatcher("result.jsp").forward(req, res); // 4. hand over to View
        } catch (Exception e) {
            throw new ServletException(e);
        }
    }
}
```

### Step 5: result.jsp (View - display only)

```jsp
<%@ page contentType="text/html;charset=UTF-8" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<html><body>
  <c:if test="${empty student}"> <h3>Roll number not found!</h3> </c:if>
  <c:if test="${not empty student}">
     <h2>${student.name}</h2>
     Marks: ${student.marks} | Grade: ${student.grade}
  </c:if>
</body></html>
```

**EXAM / INTERVIEW POINT**

**Golden rule:** do not put business logic / DB code in JSP, and do not put HTML in Servlet. That is the whole point of MVC. The Controller decides "what to do", the View decides "how it looks", the Model owns "data and rules".

## 1.4 Model 1 vs Model 2

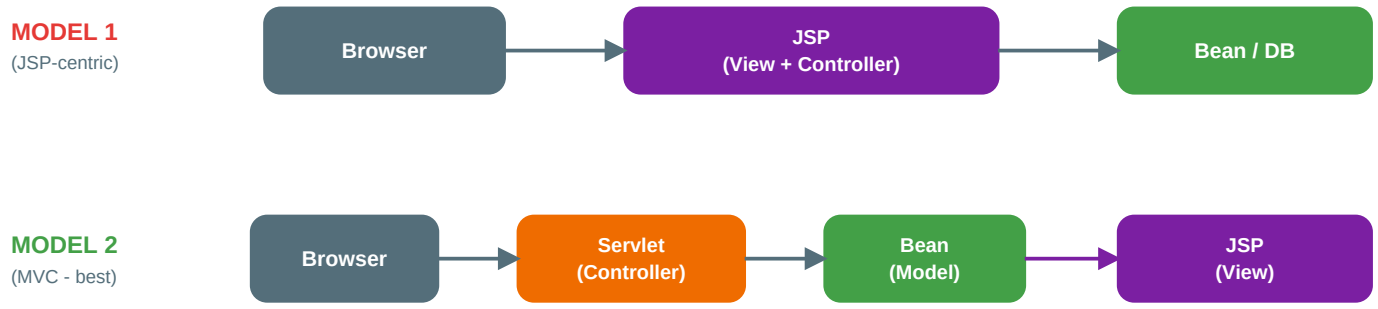



*Fig 1.2 - Model 1 (JSP handles everything) vs Model 2 (MVC)*

| Point | Model 1 | Model 2 (MVC) |
|---|---|---|
| **Controller** | The JSP itself | Servlet |
| **Complexity** | Easy for small apps | Some setup, but scalable |
| **Maintenance** | Logic + HTML mixed, hard | Separated, easy |
| **Reuse / Testing** | Low | High (bean/DAO can be tested separately) |
| **Use** | Demo / small page | Real projects, Struts, Spring MVC |

### Advantages and disadvantages of MVC

- **Advantages:** parallel development (designer works on JSP, developer on Java), code reuse, easy testing, one model with many views (web, mobile).
- **Disadvantages:** extra files/complexity for tiny apps, and the flow takes time to understand at first.
- **Frameworks:** Struts, Spring MVC and JSF all implement MVC (in Spring, controller = DispatcherServlet + @Controller).

# 2 Architecture (Java EE / Web)

Tiers, containers and a live banking example

## 2.1 What is architecture?

Application architecture describes **where the parts (UI, logic, data)** of an application run and how they talk to each other. Splitting into **tiers** (layers that can run on different machines/processes) makes an application scalable and secure.

| Type | How it works | Example | Problem |
|---|---|---|---|
| **1-Tier** | UI + logic + data in one place | MS Access / desktop app | Cannot be shared or scaled |
| **2-Tier (Client-Server)** | Client (UI+logic) <-> DB server | Swing app + JDBC directly | Logic in every client, weak security |
| **3-Tier** | Presentation <-> Business logic <-> Data | Browser + Servlet/JSP + DB | Best balance |
| **n-Tier (Java EE)** | 3-tier with separate web tier / business tier / EIS | Browser + Tomcat + EJB server + DB | Complex but highly scalable |

## 2.2 Java EE Multi-Tier Architecture

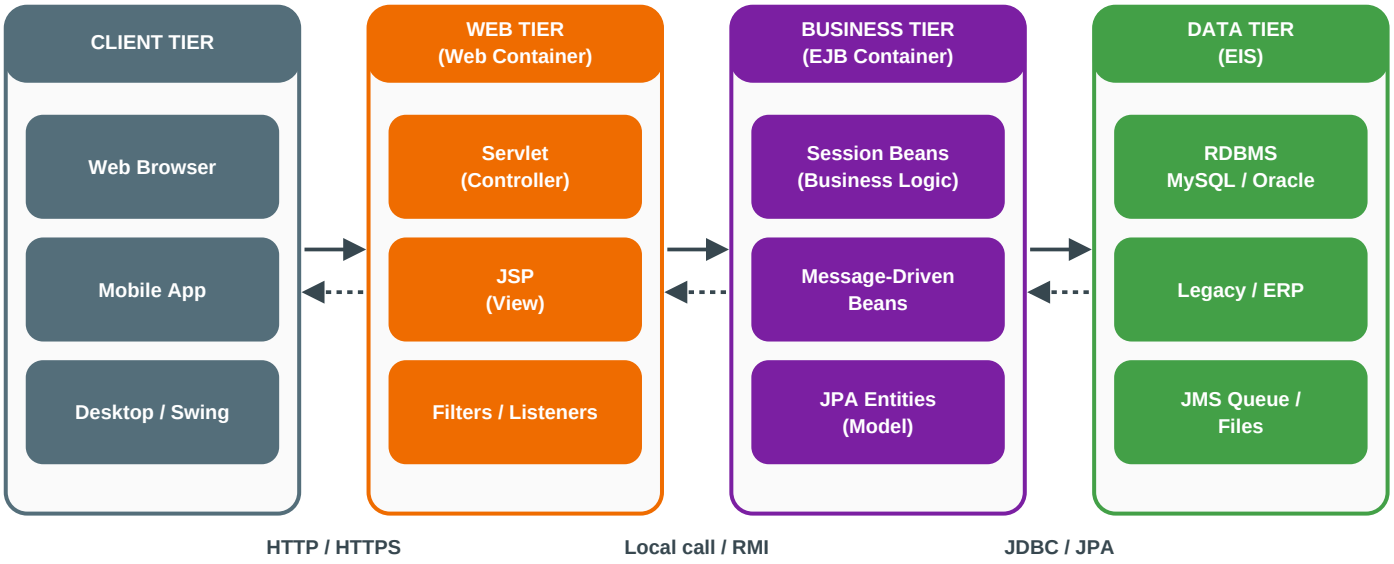



*Fig 2.1 - n-tier architecture of Java EE (Jakarta EE)*

| Client Tier | Web Tier | Business Tier | Data (EIS) Tier |
|---|---|---|---|
| Browser / mobile that sends HTTP requests and shows HTML/JSON. | Servlets, JSPs, Filters, Listeners inside the **Web Container** (Tomcat). Request handling and presentation. | Session Beans, MDB, JPA inside the **EJB Container**. Real business rules, transactions. | Database, legacy systems, message queues. Only stores data. |

## 2.3 What is a container?

A **container** is a runtime environment that manages the **lifecycle, threading, security and transactions** of components (Servlet, JSP, EJB). We write only business code; the container does the plumbing.

- **Web Container (Servlet Container):** runs Servlets + JSPs. Examples: **Apache Tomcat**, Jetty.
- **EJB Container:** runs Enterprise Beans (transactions, pooling, security).
- **Application Server:** Web + EJB + JMS + JPA together. Examples: **WildFly, GlassFish/Payara, WebLogic, WebSphere**.

**EXAM / INTERVIEW POINT**

**Tomcat is only a web container** (Servlet/JSP). EJB does not run on plain Tomcat - you need TomEE or a full application server.

## 2.4 Live Example: Online Bank - Fund Transfer

A user transfers Rs. 5000 to a friend. See what each tier does:

| Step | Tier | What happens |
|---|---|---|
| **1** | Client | User fills the form; `POST /transfer` goes over HTTP. |
| **2** | Web Container | The request reaches **TransferServlet** (Controller), which validates the amount and accounts. |
| **3** | Web -> Business | The Servlet calls `transfer(from,to,amt)` on the `@EJB TransferBean`. |
| **4** | Business (EJB) | The bean starts a transaction: debit + credit. Both succeed or both fail (atomic). The container manages the transaction. |
| **5** | Data Tier | The **accounts** table is updated through JPA/JDBC; commit. |
| **6** | Web Tier | The Servlet stores the result bean in the request and forwards to **success.jsp**. |
| **7** | Client | The browser shows the "Transfer Successful" page. |

**TIP**

**Benefit of this architecture:** to change the web tier (JSP -> React) you do not touch the business tier. To change the DB, only the data tier changes. When load grows, scale web and business servers independently.

# 3 Beans Theory (JavaBean)

Rules, properties, JSP bean tags

## 3.1 What is a bean?

A **JavaBean** is a simple **reusable Java class** that follows fixed conventions so that tools, JSP and frameworks can understand it automatically. A bean mainly **holds data** and carries it from one place to another (Servlet -> JSP).

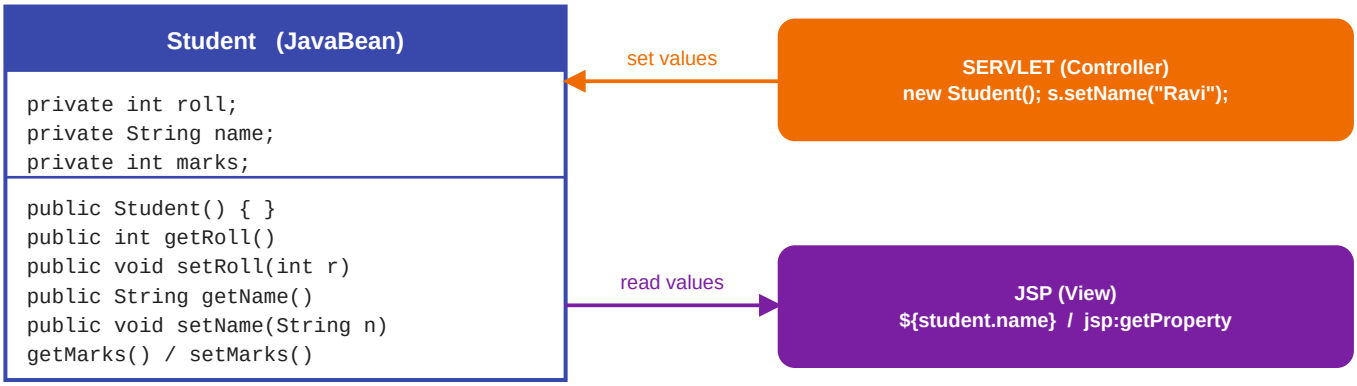



*Fig 3.1 - Structure of a JavaBean and how Servlet/JSP use it*

## 3.2 The 5 rules of a JavaBean

| # | Rule | Why? |
|---|---|---|
| 1 | **public class** | The container/JSP can access it from anywhere. |
| 2 | **public no-argument constructor** | `jsp:useBean` / reflection creates the object without arguments. |
| 3 | **private properties (fields)** | Encapsulation: data is not touched directly. |
| 4 | **public getter/setter**: `getXxx()` / `setXxx()` (`isXxx()` for boolean) | The naming convention reveals the property name: `getName()` -> property **name**. |
| 5 | **Implements Serializable** (recommended) | Needed to store in session / save to disk. |

> **WARNING**
> The property name comes from the getter, not the field. `getFirstName()` -> property **firstName**. In EL you write `${emp.firstName}`. A wrong name gives PropertyNotFoundException.

## 3.3 Types of bean properties

| Type | Meaning | Example |
|---|---|---|
| **Simple** | A single value | `String name` |
| **Indexed** | Multiple values like an array/list | `String[] hobbies`, `getHobbies(int i)` |
| **Bound** | Listeners are notified when the value changes (PropertyChangeListener) | GUI beans |
| **Constrained** | A listener can **veto** the change | Validation in GUI beans |

## 3.4 Using a bean in JSP (action tags)

JSP

```
<%-- 1. create the bean or reuse an existing one --%>
<jsp:useBean id="emp" class="com.app.model.Employee" scope="session" />

<%-- 2. fill the bean from all form fields (names must match) --%>
<jsp:setProperty name="emp" property="*" />

<%-- or one property at a time --%>
<jsp:setProperty name="emp" property="name" value="Amit" />

<%-- 3. read a value --%>
Name: <jsp:getProperty name="emp" property="name" />
Or with EL: ${emp.name}
```

**What useBean does:** it first looks for an object named by **id** in the given `scope`. If **found**, it reuses it; if **not found**, it creates a new object and stores it in that scope.

| scope | How long the bean lives | Where it is stored |
|---|---|---|
| **page (default)** | Only that JSP page | PageContext |
| **request** | One request (including forward/include) | HttpServletRequest |
| **session** | The whole session of one user | HttpSession |
| **application** | The whole web app, all users | ServletContext |

## 3.5 The different meanings of "Bean"

| Term | What it is | Runs where | Used for |
|---|---|---|---|
| **POJO** | Any plain Java class, no rules | Anywhere | General objects |
| **JavaBean** | POJO + bean rules (no-arg, getters/setters) | JVM / Web container | Data transfer, JSP, GUI |
| **Enterprise Bean (EJB)** | Server-side business component | **EJB container** | Business logic, transactions |
| **Spring Bean** | Object managed by the Spring IoC container | Spring container | Dependency injection |
| **Managed / CDI Bean** | Managed by JSF/CDI | Jakarta EE | JSF backing bean |

**EXAM / INTERVIEW POINT**
**JavaBean vs EJB:** JavaBean = small reusable class (data/UI), needs no container. EJB = heavy-weight server component that runs in an EJB container and provides transactions/security/remote access. Both have "Bean" in the name but their purpose differs.

# 4 EJB - Enterprise JavaBeans

Server-side business components

## 4.1 What is EJB?

**EJB (Enterprise JavaBeans)** is the **server-side component model** of Java EE where we write **business logic**. An EJB runs inside an **EJB container**, and the container automatically provides heavy features like **transactions, security, pooling, concurrency and remote access**.

## 4.2 Why EJB? (problems without it)

- Every project would have to hand-write transaction commit/rollback, security checks and connection pooling.
- Stuffing business logic into servlets mixes code and prevents reuse.
- Calling a method on a remote machine (distributed app) would need manual sockets/RMI.
- With EJB you just add an **annotation** (`@Stateless`) and the container handles the rest.

## 4.3 EJB Architecture

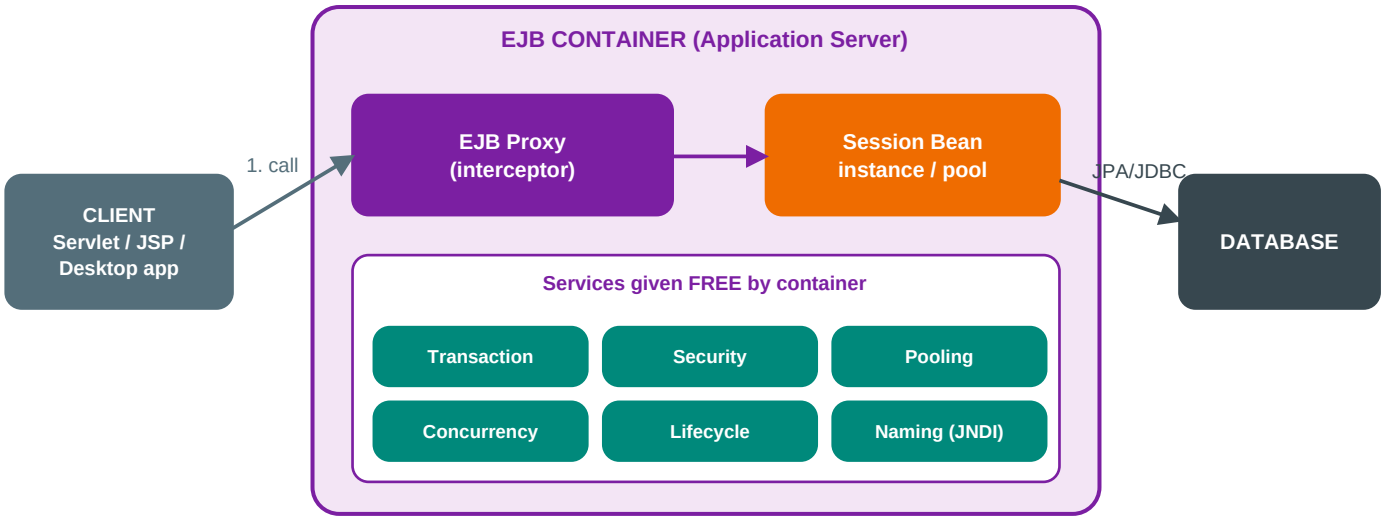



*Fig 4.1 - The client never talks to the bean directly; a container proxy sits in between*

When a client calls an EJB it first gets a **proxy/stub**. Through this proxy the container runs **interceptors**: start transaction, security check, take a bean instance from the pool, run the method, commit, return the bean to the pool.

## 4.4 Types of EJB

| Type | Job | When to use |
|---|---|---|
| **Session Bean (Stateless / Stateful / Singleton)** | Performs a business operation for a client (synchronous) | Transfer, place order, calculate interest |
| **Message-Driven Bean (MDB)** | Listens to a JMS queue/topic message **asynchronously**. The client never gets a bean reference | Sending email, processing an order in the background |
| **Entity Bean (old, EJB 2.x)** | Used to represent a DB row | Now **deprecated**; replaced by **JPA @Entity** |

## 4.5 Free services from the container

- **Transaction management** - `@TransactionAttribute`, automatic commit/rollback.
- **Security** - `@RolesAllowed("ADMIN")`.
- **Instance pooling** - bean objects are reused, not created every time.
- **Concurrency** - the container handles thread-safety (stateless/stateful).

- **Dependency injection** - `@EJB`, `@PersistenceContext`.
- **Remote access** - calls from another JVM/machine (RMI/IIOP).
- **Timer service, interceptors, lifecycle callbacks**.

### Local vs Remote interface

| | @Local (or no-interface) | @Remote |
|---|---|---|
| **Client** | Same JVM / same application (Servlet -> EJB) | Another JVM or another server |
| **Speed** | Fast (direct reference, pass-by-reference) | Slower (network, pass-by-value, serialization) |
| **Use** | Most web apps | Distributed systems / remote clients |

## 4.6 Code Example: Stateless EJB + Servlet

**Java**

```
// ---- Business logic (EJB) ----
@Stateless
public class InterestBean {
    public double simpleInterest(double p, double r, double t) {
        return (p * r * t) / 100;
    }
}
```

**Java**

```
// ---- Controller (Servlet) ----
@WebServlet("/si")
public class SIServlet extends HttpServlet {
    @EJB                                    // the container injects the bean (do not use new!)
    private InterestBean bean;

    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
        double si = bean.simpleInterest(10000, 8, 2);
        res.getWriter().println("Simple Interest = " + si);     // 1600.0
    }
}
```

**Java**

```
// ---- Message-Driven Bean (asynchronous) ----
@MessageDriven(mappedName = "jms/OrderQueue")
public class OrderMDB implements MessageListener {
    public void onMessage(Message m) {
        // the container calls this method as soon as a message arrives in the queue
    }
}
```

**WARNING**

Never create an EJB with `new InterestBean()`. You would lose the container services (transaction, security). Always use `@EJB` / JNDI lookup.

**EXAM / INTERVIEW POINT**

**Advantages of EJB:** rapid development, scalability, built-in transaction/security, portability. **Disadvantages:** heavy, needs an application server, overkill for small apps (that is why Spring became popular; modern EJB 3.x/4.0 is lightweight).

# 5 Session & Session Tracking

HTTP is stateless, so how do we remember the user?

## 5.1 The problem: HTTP is stateless

The HTTP protocol is **stateless** - every request is **new and independent**, and the server does not remember the previous one. Because of this, login, shopping carts and preferences would not work. The solution is **Session Tracking**.

> **NOTE**
> **Session** = a **series of related requests** made by the same user from the same browser (from login until logout/timeout). The server keeps a separate **session object** in memory for each user, holding that user's data.

## 5.2 4 ways of session tracking

| Technique | How it works | Advantage | Disadvantage |
|---|---|---|---|
| **Cookies** | Server sends `Set-Cookie`, browser returns it with every request | Simple, automatic | User can disable them, size limit (~4KB) |
| **URL Rewriting** | ID appended to the URL: `page;jsessionid=ABC123` (`response.encodeURL()`) | Works even if cookies are disabled | Long URL, risk of ID leak |
| **Hidden Form Fields** | Data inside `<input type="hidden" ...>` | No cookies needed | Only on form submit; visible in page source |
| **HttpSession** | Server-side object; the ID travels by cookie or URL | Secure, large data, easiest | Uses server memory |

## 5.3 How HttpSession works

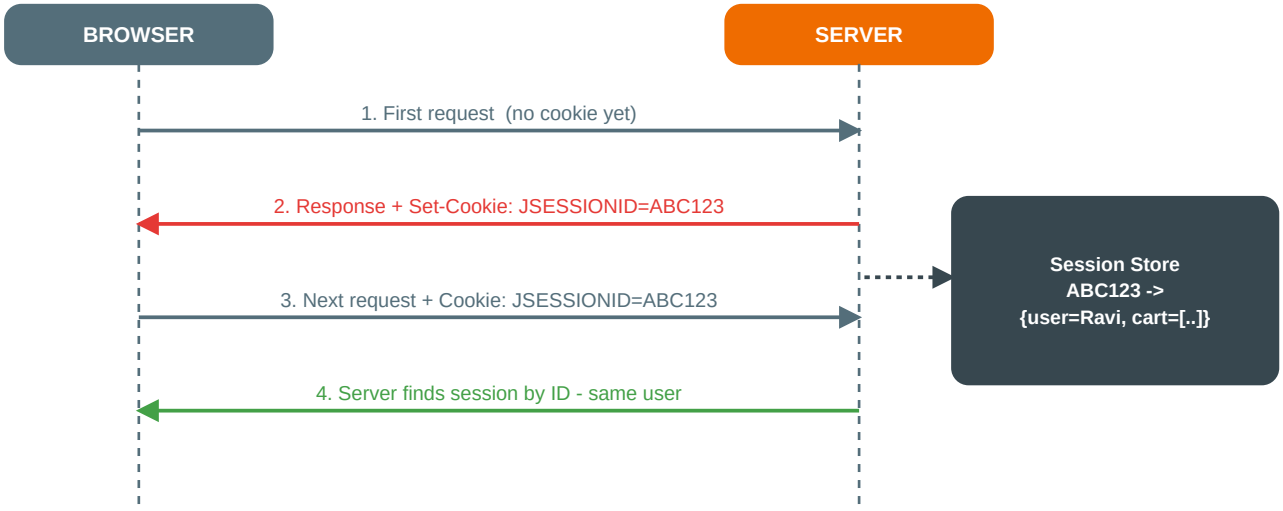



*Fig 5.1 - The server recognises the user through the JSESSIONID cookie*

## 5.4 Important HttpSession methods

| Method | Job |
|---|---|
| `request.getSession()` | Get the session; **create a new one if none exists** |
| `request.getSession(false)` | Get only an existing one; **returns null if none** (best for login checks) |
| `setAttribute(name, obj)` / `getAttribute(name)` | Store / retrieve data (returns Object - cast it) |
| `removeAttribute(name)` | Remove one attribute |
| `invalidate()` | Destroy the whole session (logout) |
| `setMaxInactiveInterval(sec)` | Idle timeout in seconds (-1 = never) |
| `getId()` / `isNew()` | Session ID / is it new? |
| `getCreationTime()` / `getLastAccessedTime()` | Time info (milliseconds) |

## 5.5 Live Example: Login -> Dashboard -> Logout

Java

```
// LoginServlet
protected void doPost(HttpServletRequest req, HttpServletResponse res) throws IOException {
    String u = req.getParameter("user"), p = req.getParameter("pass");
    if (new UserDAO().isValid(u, p)) {
        req.changeSessionId();                          // protects against session fixation
        HttpSession session = req.getSession();         // new session
        session.setAttribute("user", u);                // store data
        session.setMaxInactiveInterval(15 * 60);        // 15 min idle timeout
        res.sendRedirect("dashboard");
    } else {
        res.sendRedirect("login.html?error=1");
    }
}
```

Java

```
// DashboardServlet - protected page
protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
    HttpSession session = req.getSession(false);       // do not create a new session
    if (session == null || session.getAttribute("user") == null) {
        res.sendRedirect("login.html");                // not logged in
        return;
    }
    String user = (String) session.getAttribute("user");
    res.getWriter().println("Welcome " + user);
}

// LogoutServlet
HttpSession s = req.getSession(false);
if (s != null) s.invalidate();                         // end the session
res.sendRedirect("login.html");
```

XML

```
<!-- web.xml : timeout (minutes) -->
<session-config>
    <session-timeout>30</session-timeout>
</session-config>
```

**TIP**

**Security tips:** give a new session ID after login (`changeSessionId()`), keep the cookie **HttpOnly + Secure**, do not put very large objects in the session, and never store sensitive data (passwords) in it.

# 6 Session Beans (EJB)

Stateless | Stateful | Singleton

## 6.1 What is a Session Bean?

A **Session Bean** is an EJB that performs a **business task on behalf of one client** (such as transferring money or managing a cart). It is called "session" because it represents **one interaction/session** of the client.

> **WARNING**
> **Do not confuse: HttpSession** (Servlet API, web tier, holds user data) and a **Session Bean** (EJB, business tier, holds business logic) are **different things**. They just share the word "session".

## 6.2 Three types

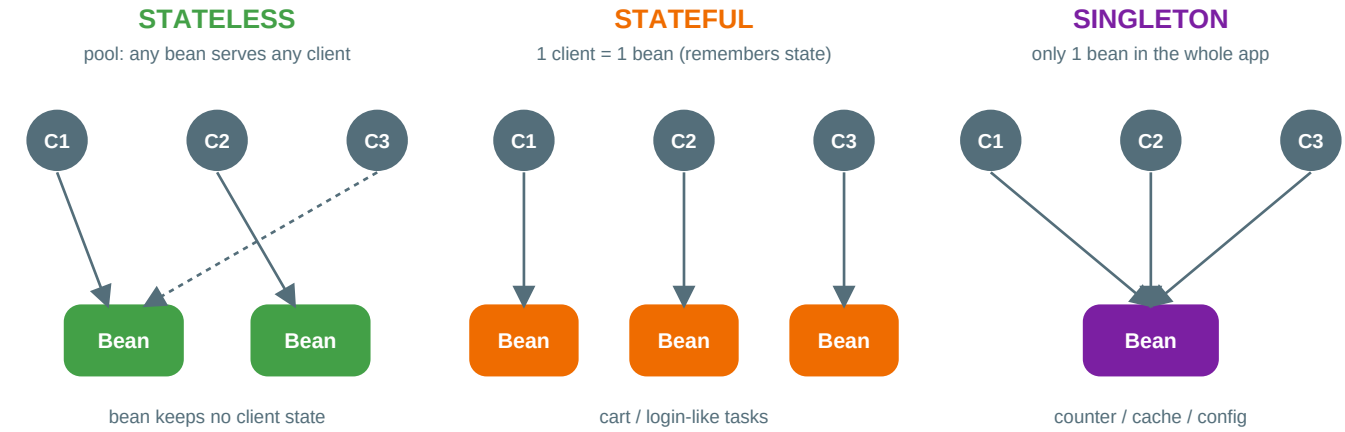



*Fig 6.1 - Relationship between clients and bean instances (C = client)*

| Point | Stateless | Stateful | Singleton |
|---|---|---|---|
| **Annotation** | `@Stateless` | `@Stateful` | `@Singleton` |
| **State** | Does not remember the client conversation | Remembers client-specific state (instance variables) | Shared state for the whole app |
| **Instances** | Many in a pool; any one serves any call | 1 client = 1 instance | Only **1** in the container |
| **Concurrency** | Container gives one thread per bean at a time | Same | Container-managed (`@Lock` READ/WRITE) or bean-managed |
| **Performance** | Fastest and most scalable | More memory (a bean per client) | Fast, but lock contention |
| **Passivation** | No | **Yes** (idle bean saved to disk) | No |
| **Use case** | Calculator, payment, email service | Shopping cart, multi-step wizard | Counter, cache, app config |
| **Web equivalent** | (utility service) | Like HttpSession | Like ServletContext |

## 6.3 Lifecycle of a Stateful bean




*Fig 6.2 - Stateful session bean life cycle*

| Callback | When it runs |
|---|---|
| `@PostConstruct` | Right after the bean is created and injected (initialisation work) |
| `@PrePassivate` | Before an idle bean is removed from memory and written to disk (close resources) |
| `@PostActivate` | After it is brought back from disk (reopen resources) |
| `@Remove` | When this method is called the container will destroy the bean (checkout/logout) |
| `@PreDestroy` | Just before the bean is destroyed (cleanup) |

**NOTE**

**Stateless/Singleton lifecycle:** Does Not Exist -> Ready (@PostConstruct) -> Does Not Exist (@PreDestroy). The Passive state exists only for **stateful** beans.

## 6.4 Code Examples

Java

```
@Stateful
public class CartBean implements Serializable {
    private List<String> items = new ArrayList<>();       // client-specific state

    public void add(String item)     { items.add(item); }
    public List<String> getItems()   { return items; }

    @Remove
    public void checkout() { /* save order to DB */ items.clear(); }

    @PostConstruct void init()    { System.out.println("Cart ready"); }
    @PreDestroy   void cleanup()  { System.out.println("Cart destroyed"); }
}
```

Java

```
@Singleton
@Startup                                          // bean is created as soon as the app starts
public class VisitorCounterBean {
    private int count;

    @Lock(LockType.WRITE)  public int hit() { return ++count; }   // one thread at a time
    @Lock(LockType.READ)   public int get() { return count; }     // many readers together
}
```

**EXAM / INTERVIEW POINT**

**Which one when?** Need to remember state (cart) -> **Stateful**. Just do work with no state (calculation, payment) -> **Stateless** (default choice). One shared object for the whole app (counter/cache) -> **Singleton**.

# 7 Servlet Session vs JSP Session

Are they different? The full truth

## 7.1 The truth first

**EXAM / INTERVIEW POINT**

**Servlet session and JSP session are the same thing - both are objects of** `javax.servlet.http.HttpSession`**.** A JSP is internally converted into a servlet, and inside that servlet's `_jspService()` the container automatically creates a variable named `session` (an implicit object). So data set in a servlet is available in the JSP, and data set in the JSP is available in the servlet.

**Java**

```
// Inside _jspService() of the translated servlet (written by the container):
HttpSession session = null;
...
session = pageContext.getSession();     // <-- this is the JSP "session" implicit object
```

## 7.2 Difference Table

| Point | Session in Servlet | Session in JSP |
|---|---|---|
| **Object type** | HttpSession | HttpSession (same) |
| **How you get it** | You must ask for it: `request.getSession()` | It is an **implicit object**: just write `session` |
| **Creation** | Explicit: `getSession(true)` creates; `getSession(false)` does not (null) | Automatically created/obtained when page directive has `session="true"` (default) |
| **Default behaviour** | No session is created until you call getSession() | Every JSP visit may create a session (default session=true) |
| **Disabling** | Do not call getSession() / use `getSession(false)` | `<%@ page session="false" %>`; then the `session` variable is unavailable (compile error) |
| **Null check** | Required (`getSession(false)` can return null) | Normally never null (with session=true) |
| **Code style** | In a Java class | In scriptlet/EL: `${sessionScope.user}` |
| **Typical use** | Login/logic: set, validate, invalidate the session | **Display** the session data |
| **Scope** | Session scope | Session scope (same) |
| **Data sharing** | Set in servlet -> visible in JSP | Set in JSP -> visible in servlet |

## 7.3 Live demo: servlet sets, JSP reads

**Java**

```
// LoginServlet.java
HttpSession session = request.getSession();           // explicit in servlet
session.setAttribute("user", "Ravi");
response.sendRedirect("welcome.jsp");
```

**JSP**

```
<%-- welcome.jsp  (session implicit object) --%>
<%
    String u = (String) session.getAttribute("user");  // same HttpSession object!
    out.println("Hello " + u);                          // Hello Ravi
%>
<%-- with EL --%>
Hello ${sessionScope.user}
```

**WARNING**

**Hidden cost:** JSP has `session="true"` by default, so opening any JSP page may make the server create a session. On public pages (home, about, static info) add `<%@ page session="false" %>` to save memory.

# 8 JSP vs Servlet

The complete difference

## 8.1 In one line

**Servlet** = writing HTML inside Java code. **JSP** = writing Java inside HTML. Both produce dynamic web content, and a JSP finally becomes a Servlet.

> **EXAM / INTERVIEW POINT**
> **Final answer:** Servlet for logic/control, JSP for presentation. Real projects use both: **Servlet (Controller) -> JSP (View)**.

## 8.2 13-point Difference

| # | Point | Servlet | JSP |
|---|---|---|---|
| 1 | Definition | A Java class that handles requests | A text-based page with HTML + Java tags |
| 2 | Code mix | HTML **inside Java** (`out.println("<h1>")`) | Java **inside HTML** (`<%= %>`) |
| 3 | Best for | Controller / business logic | View / presentation |
| 4 | MVC role | Controller | View |
| 5 | Ease | Hard (HTML strings, quotes) | Easy; a designer can work on it too |
| 6 | Compilation | Developer compiles (.class) | Container translates + compiles it |
| 7 | On change | Recompile + redeploy/reload | Save the file; updated on the next request |
| 8 | Extension / file | `.java` / `.class` | `.jsp` |
| 9 | Implicit objects | None; you create `request.getSession()`, `getServletContext()` yourself | 9 implicit objects ready |
| 10 | Tag libraries / EL | Not supported | JSTL, custom tags, EL supported |
| 11 | Life cycle | init(), service(), destroy() | jspInit(), _jspService(), jspDestroy() (+ translation, compile) |
| 12 | Mapping | `@WebServlet` / mapping in web.xml | Direct URL (`/page.jsp`), mapping not required |
| 13 | Extends | You extend `HttpServlet` | The generated class extends `HttpJspBase` |

## 8.3 Same output, both ways

**Java**

```
// Servlet version
public class HelloServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        out.println("<html><body><h1>Hello " + req.getParameter("name") + "</h1>");
        out.println("<p>Time: " + new java.util.Date() + "</p>");
        out.println("</body></html>");
    }
}
```

**JSP**

```
<%-- JSP version: same output, much shorter --%>
<html><body>
   <h1>Hello <%= request.getParameter("name") %></h1>
   <p>Time: <%= new java.util.Date() %></p>
</body></html>
```

# 9 Servlet & JSP Life Cycle

init - service - destroy | jspInit - _jspService - jspDestroy

## 9.1 Servlet Life Cycle

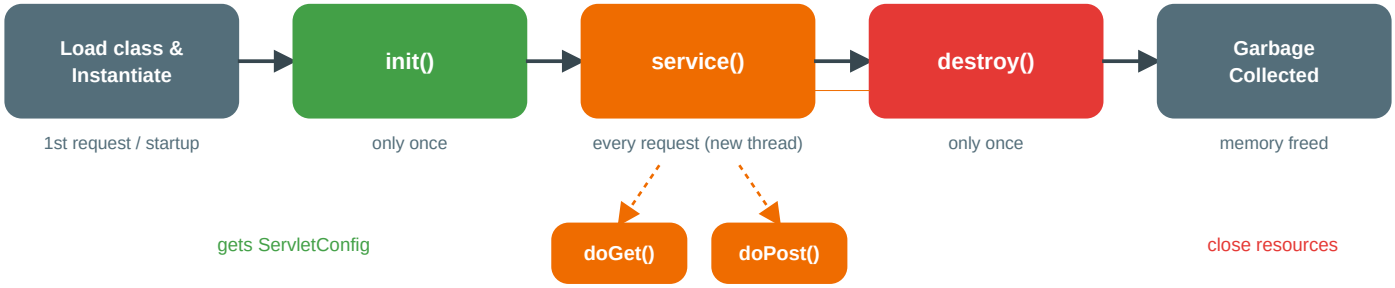



*Fig 9.1 - Servlet lifecycle: the container controls everything*

| Phase | Who does it | What happens | How often |
|---|---|---|---|
| **Loading + Instantiation** | Container | The class is loaded and **only one object** is created with the default constructor (on the 1st request, or at startup if `load-on-startup` is set) | 1 |
| `init(ServletConfig)` | Container | Initialisation: DB connection, reading config. Requests are served only after it completes | 1 |
| `service(req,res)` | Container | For every request a **new thread** runs on this same object. HttpServlet's service() checks GET/POST and calls `doGet()/doPost()` | Every request |
| `destroy()` | Container | On server shutdown / app undeploy. Close resources, save data | 1 |

Java

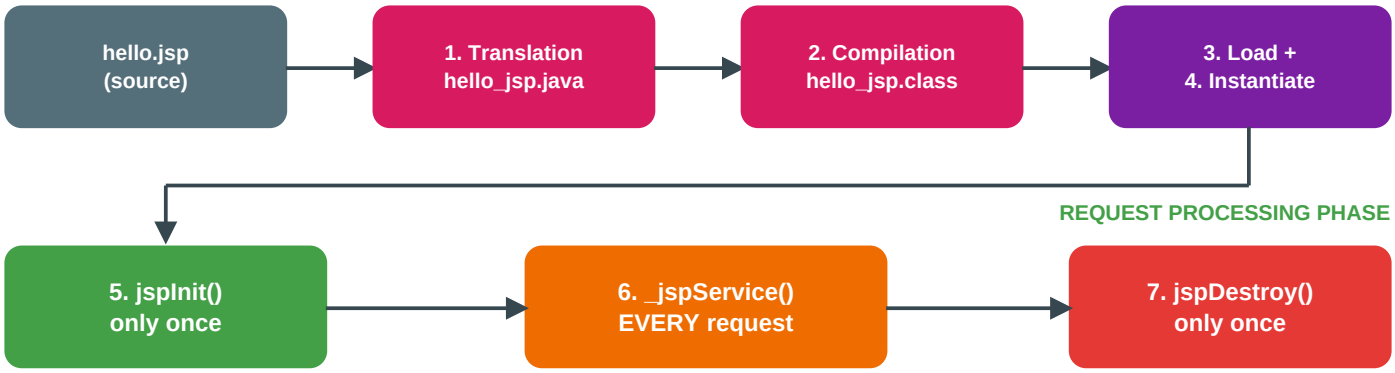
```
public class LifeServlet extends HttpServlet {
    public void init(ServletConfig cfg) throws ServletException {
        super.init(cfg);
        System.out.println("init()    - only once");
    }
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
        System.out.println("service -> doGet() - on every request");
        res.getWriter().println("Hello");
    }
    public void destroy() {
        System.out.println("destroy() - only once");
    }
}
```

**WARNING**

**Thread-safety:** A servlet has **only one object** shared by many threads. So **do not keep user-specific data in servlet instance variables** (race condition). Local variables are safe.

## 9.2 JSP Life Cycle

**TRANSLATION PHASE (first request, or when the JSP changes)**




**Mnemonic: T - C - L - I - I - R - D**

(Translate, Compile, Load, Instantiate, Init, Request, Destroy)

*Fig 9.2 - JSP lifecycle: 7 phases (Translation to Destroy)*

| Phase | Detail |
|---|---|
| **1. Translation** | The JSP file is converted into **servlet source (hello_jsp.java)**. JSP syntax errors are caught here. |
| **2. Compilation** | hello_jsp.java -> **hello_jsp.class**. Java syntax errors appear here. |
| **3. Class loading** | The class is loaded into the JVM. |
| **4. Instantiation** | **One instance** of the generated servlet is created. |
| **5. Initialization -** `jspInit()` | **Once** only. Resources/connections go here. |
| **6. Request processing -** `_jspService()` | On **every request**. All the HTML + scriptlet code runs here. |
| **7. Destroy -** `jspDestroy()` | Once, when the JSP is removed / server shuts down. Cleanup. |

### The three JSP life-cycle methods in detail

| Method | Can you override it? | When | Job |
|---|---|---|---|
| `public void jspInit()` | Yes (inside a declaration <%! %>) | Once, before the first request | Initialisation: DB connection, load config |
| `public void _jspService(HttpServletRequest, HttpServletResponse)` | **No** - the container generates it (hence the underscore) | Every request | The whole JSP body (HTML + scriptlet) goes into this method |
| `public void jspDestroy()` | Yes | Once, before destruction | Cleanup: close connections, files |

**JSP**

```
<%@ page import="java.sql.*" %>
<%!
    Connection con;                              // instance variable (shared!)

    public void jspInit() {                      // once
        System.out.println("jspInit() called");
        // con = DriverManager.getConnection(...);
    }
    public void jspDestroy() {                   // once
        System.out.println("jspDestroy() called");
        // con.close();
    }
%>
<%
    out.println("_jspService() is running - on every request");   // part of _jspService
%>
```

### The servlet the container generates (simplified)

```java
public final class hello_jsp extends HttpJspBase {
    int visits = 0;                                  // <%! %> declarations land here
    public void jspInit()    { ... }                 // your override
    public void jspDestroy() { ... }

    public void _jspService(HttpServletRequest request, HttpServletResponse response)
            throws IOException, ServletException {
        HttpSession session = null;  ServletContext application = null;     // implicit
        ServletConfig config = null; JspWriter out = null; Object page = this;
        PageContext pageContext = null;
        ...
        out.write("<html><body>");                   // template text
        out.print( request.getParameter("name") );   // <%= %> expression
        out.write("</body></html>");
    }
}
```

## 9.3 Servlet vs JSP lifecycle - at a glance

| Servlet | JSP | How often |
|---|---|---|
| **(developer compiles)** | Translation + Compilation (container) | First request / on change |
| `init()` | `jspInit()` | Once |
| `service() -> doGet/doPost` | `_jspService()` | Every request |
| `destroy()` | `jspDestroy()` | Once |

**EXAM / INTERVIEW POINT**
**Memory trick:** JSP lifecycle = **T-C-L-I-I-R-D** (Translate, Compile, Load, Instantiate, Init, Request, Destroy). And you cannot override `_jspService()` because it is generated from the JSP body.

# 10 What is JSP & Why JSP

The complete concept of JavaServer Pages

## 10.1 What is JSP?

**JSP (JavaServer Pages)** is a **server-side technology** in which we embed **Java code inside HTML** to create **dynamic web pages**. The file extension is `.jsp`. The container **translates the page into a Servlet** and runs it, so a JSP is really an "easier form of servlet".

## 10.2 Problems of Servlets -> birth of JSP

- In a servlet, HTML must be written with `out.println("...")` - long, quote-heavy and hard to read.
- A web designer does not know Java and cannot edit a servlet.
- Even a small text/colour change needs recompile + redeploy.
- Presentation and logic get mixed.

## 10.3 Why JSP - Advantages

| Advantage | Explanation |
|---|---|
| **Easy to write** | Write it like HTML and use <%= %> / EL where dynamic data is needed. |
| **Separation of roles** | Designer does the HTML part, developer does Java/beans. |
| **Auto compile** | Change and save; the next request uses the new version (no redeploy). |
| **Implicit objects** | request, session, out... without declaring them. |
| **Tag libraries** | JSTL and custom tags reduce scriptlets and keep code clean. |
| **Full Java power** | JDBC, collections, beans, EJB - all usable. |
| **Platform independent** | Write once, run anywhere (JVM). |
| **Fast at runtime** | After compilation it runs at servlet speed. |
| **Part of Java EE** | Integrates with Servlet, JSTL, JSF, EL. |

## 10.4 Anatomy of a JSP page

JSP

```
<%@ page language="java" contentType="text/html; charset=UTF-8" %>   <!-- 1. Directive -->
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>      <!-- 1. Directive -->
<%! int count = 0; %>                                                 <!-- 2. Declaration -->
<html>
<body>                                                                <!-- 3. Template text (HTML) -->
   <% count++; String user = request.getParameter("u"); %>            <!-- 4. Scriptlet -->
   <h1>Welcome <%= user %></h1>                                       <!-- 5. Expression -->
   <p>Visit No: ${count}</p>                                          <!-- 6. EL -->
   <c:if test="${not empty param.u}">Hello!</c:if>                    <!-- 7. JSTL -->
   <jsp:include page="footer.jsp" />                                  <!-- 8. Action tag -->
</body>
</html>
```

> **NOTE**
> **JSP = Template text (HTML) + JSP elements.** Template text goes to the response as it is; JSP elements execute on the server.

## 10.5 Limitations of JSP

- Debugging is harder than with servlets (line numbers refer to the translated file).
- Too many scriptlets turn the code into spaghetti - so use MVC + JSTL/EL.
- A small delay on the first request because of translation + compilation.

- Heavy business logic should not be written in JSP.

> **TIP**
> **Modern practice:** avoid scriptlets (<% %>) in JSP; use EL (${}) + JSTL and keep logic in Servlet/Service. Thymeleaf/JSF/React are also used for views today, but JSP is still very common in syllabi and legacy projects.

# 11 Which is Faster: JSP or Servlet?

A direct answer and the reasons

## 11.1 Answer

**EXAM / INTERVIEW POINT**

**At runtime both are almost equally fast** (a JSP finally becomes a servlet and the same .class runs). **On the first request JSP is slower** (translation + compilation). **A Servlet is considered slightly faster** because it has no such extra step. **In development, JSP is faster.**

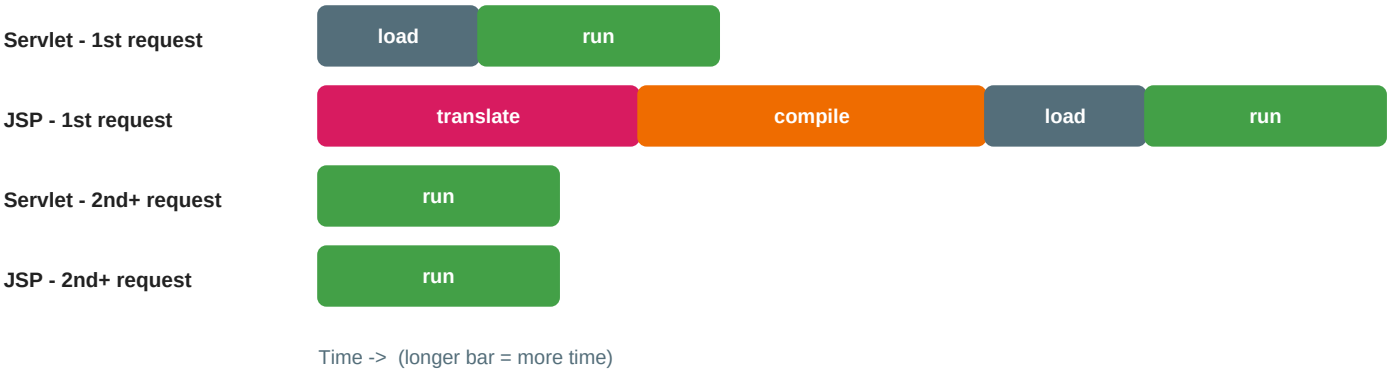



*Fig 11.1 - On the first request a JSP must also translate + compile; afterwards both are equal*

## 11.2 Comparison

| Factor | Servlet | JSP |
|---|---|---|
| **First request** | Class load + init | **Translate + compile + load + init** (slow) |
| **2nd request onwards** | Fast | Fast (same as servlet) |
| **Generated code** | What you wrote | Container-generated code (a few extra out.write calls) |
| **Development speed** | Slow (HTML strings) | **Fast** |
| **Maintenance** | Changing presentation is hard | Easy |
| **Overall winner** | Tiny runtime advantage | Productivity advantage |

## 11.3 Tips to make JSP faster

- **Precompile** JSPs (Tomcat `jspc` / build time) so the first request is not slow.
- Use `<%@ page session="false" %>` where no session is needed.
- Use `<%@ include %>` (compile time) for static content; keep heavy loops/logic in the servlet/service.
- Connection **pooling** (DataSource), `PreparedStatement`, caching.
- Tune the output **buffer** (`buffer="16kb"`), use gzip compression, avoid large objects in the session.

**TIP**

**Interview line:** "The speed difference is negligible. After it is compiled once, a JSP runs like a servlet. If you must choose: servlet for logic, JSP for the view."

# 12 9 Implicit Objects of JSP

Ready-made objects given by the container

## 12.1 What are implicit objects?

Inside a JSP you can use some objects **without declaring or creating them**. The container **creates them automatically** inside the generated servlet's `_jspService()`. There are **9** in total.

> **WARNING**
> Implicit objects are available only in **scriptlets** <% %> **and expressions** <%= %>, **not in declarations** <%! %> (because those go to class level, outside _jspService).

| # | Object | Type (class) | Scope | Servlet equivalent |
|---|---|---|---|---|
| 1 | **request** | HttpServletRequest | request | req of doGet/doPost |
| 2 | **response** | HttpServletResponse | page | res of doGet/doPost |
| 3 | **out** | JspWriter | page | res.getWriter() |
| 4 | **session** | HttpSession | session | req.getSession() |
| 5 | **application** | ServletContext | application | getServletContext() |
| 6 | **config** | ServletConfig | page | getServletConfig() |
| 7 | **pageContext** | PageContext | page | (JSP-specific) |
| 8 | **page** | Object (this) | page | this |
| 9 | **exception** | Throwable | page | (error page only) |

## 12.2 Scope diagram

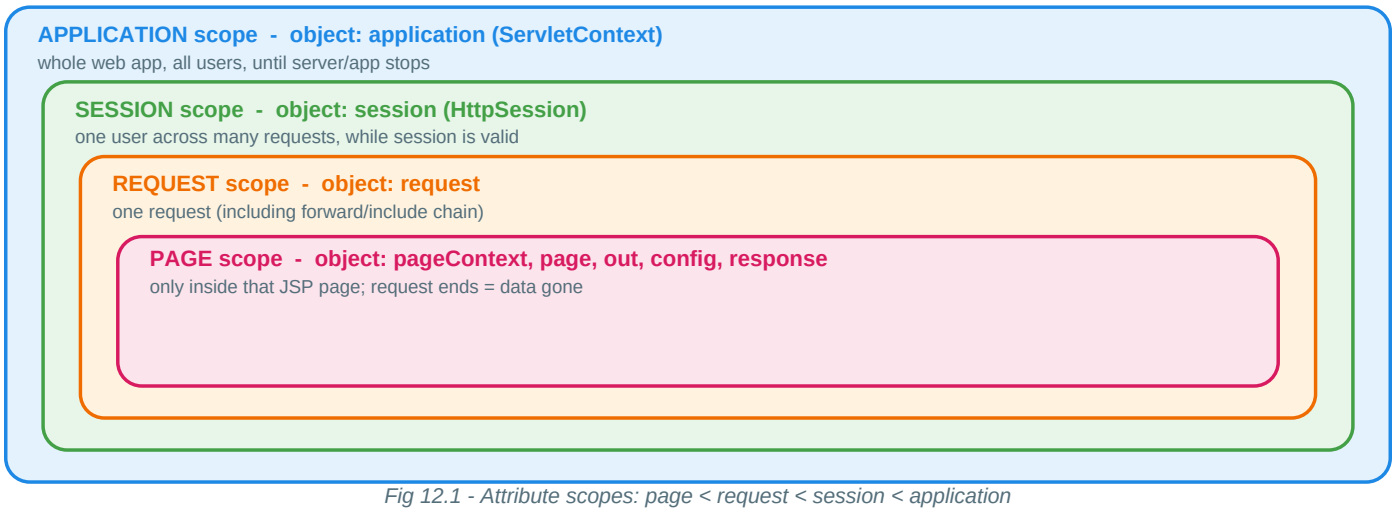



*Fig 12.1 - Attribute scopes: page < request < session < application*

## 12.3 Each object in detail

### 1. request `HttpServletRequest`

Data of the client's request: form parameters, headers, cookies, IP. You can also keep attributes in **request scope**.

```
String user = request.getParameter("user");

request.setAttribute("msg", "Hi");

String ip = request.getRemoteAddr();
```

### 2. response — `HttpServletResponse`

Controls the response going to the client: redirect, content type, cookie, header.

```
response.sendRedirect("home.jsp");

response.setContentType("text/html");

response.addCookie(new Cookie("lang", "en"));
```

### 3. out — `JspWriter`

Buffered writer to write text into the response body (like PrintWriter, but with a buffer).

```
out.println("Hello " + name);

out.flush();   // send the buffer
```

### 4. session — `HttpSession`

Shares data between multiple requests of one user (login, cart).

```
session.setAttribute("user", "Rahul");

String u = (String) session.getAttribute("user");

session.invalidate();
```

### 5. application — `ServletContext`

Shared object of the whole web app: all users, all pages. Global counter, context-param, real path.

```
application.setAttribute("hits", 10);

String db = application.getInitParameter("dbUrl");   // web.xml context-param

String path = application.getRealPath("/");
```

### 6. config — `ServletConfig`

Configuration of this JSP/servlet (init-params from web.xml, servlet name).

```
String theme = config.getInitParameter("theme");

String nm = config.getServletName();
```

### 7. pageContext — `PageContext`

The boss of all: gives access to **all other implicit objects** + set/get attributes in **any scope**.

```
pageContext.setAttribute("x", "1", PageContext.SESSION_SCOPE);

Object v = pageContext.findAttribute("x");   // page->request->session->app

pageContext.forward("a.jsp");   pageContext.getRequest();
```

### 8. page — `java.lang.Object`

The instance of the current JSP's generated servlet (a reference to **this**). Rarely used.

```
Object p = page;

out.println(page.getClass().getName());
```

### 9. exception — `java.lang.Throwable`

Available only in an **error page** (`isErrorPage="true"`). The exception object from the previous page.

```
<%@ page isErrorPage="true" %>

<%= exception.getMessage() %>

<%-- source page: <%@ page errorPage="error.jsp" %> --%>
```

**EXAM / INTERVIEW POINT**

**Memory trick: R R O S A C P P E** = **R**equest, **R**esponse, **O**ut, **S**ession, **A**pplication, **C**onfig, **P**ageContext, **P**age, **E**xception. Except **exception**, the other 8 are available in every JSP.

# 13 JSP Elements & Scriptlet

What you can write inside a JSP

## 13.1 The family of JSP elements

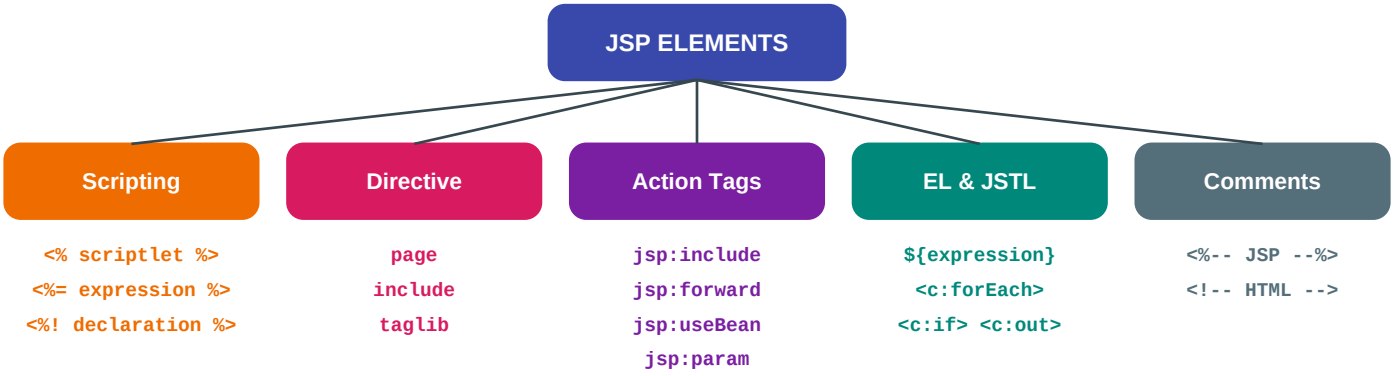



*Fig 13.1 - All JSP elements*

| Element | Syntax | Where it goes in the translated servlet | Job |
|---|---|---|---|
| **Scriptlet** | `<% code; %>` | Inside `_jspService()` (local) | Java statements, loops, conditions |
| **Expression** | `<%= expr %>` | `out.print(expr);` | Print a value to the output (no semicolon) |
| **Declaration** | `<%! decl %>` | Class level (**outside _jspService**) | Declare variables and methods |
| **Directive** | `<%@ ... %>` | Instruction to the container at translation time | page, include, taglib |
| **Action tag** | `<jsp:xxx />` | Works at runtime (method calls) | include, forward, useBean... |
| **Comment** | `<%-- --%>` | Not translated at all | Hidden comment |
| **EL / JSTL** | `${ }`, `<c:xxx>` | Evaluated at runtime | Scriptlet-free clean code |

## 13.2 Scriptlet <% ... %>

In a scriptlet you can write **any valid Java statements**. This code goes into `_jspService()`, so variables declared in it are **local** (new for every request). You **cannot declare methods** in a scriptlet (a method cannot contain a method).

**JSP**

```
<%
    int marks = Integer.parseInt(request.getParameter("m"));
    String grade;
    if (marks >= 75) grade = "A"; else grade = "B";
%>
<p>Grade: <%= grade %></p>

<% for (int i = 1; i <= 3; i++) { %>       <%-- you can break out into HTML in the middle of a loop --%>
    <li>Item <%= i %></li>
<% } %>
```

## 13.3 Expression <%= ... %>

- Converts the result to a **String** and writes it straight to the output: `out.print(expression)`.
- **Do not put a semicolon (;)** - <%= x; %> is wrong (it becomes `out.print(x;);`).
- One value per expression: <%= a + b %>, <%= new java.util.Date() %>, <%= user.toUpperCase() %>.

## 13.4 Declaration <%! ... %>

Declares variables and **methods** at class level. They become **instance variables/methods** of the generated class, so they are **shared by all requests (threads)**.

JSP

```
<%!
    int visits = 0;                                  // instance variable - SHARED by all users
    String greet(String n) { return "Hello " + n; }  // method
    public void jspInit() { /* once */ }             // lifecycle method override
%>
```

> **WARNING**
> **Thread-safety:** `<%! int visits = 0; %>` is one single variable for all users - several users at once can cause a race condition. Keep per-request data in a **local** scriptlet variable. Also, implicit objects (request, session) are not available inside `<%! %>`.

### Scriptlet vs Expression vs Declaration

| Point | Scriptlet | Expression | Declaration |
|---|---|---|---|
| **Tag** | `<% %>` | `<%= %>` | `<%! %>` |
| **Goes to** | Inside _jspService() | Inside out.print() | Class level |
| **Variable type** | Local | - | Instance (shared) |
| **Methods** | Cannot create | No | **Yes** |
| **Semicolon** | Required | **No** | Required |
| **Implicit objects** | Available | Available | **Not available** |

## 13.5 Comments

| Type | Syntax | Visible to the client? |
|---|---|---|
| **JSP comment** | `<%-- note --%>` | **No** (not even translated) |
| **HTML comment** | `<!-- note -->` | **Yes** (visible in view-source) |
| **Java comment** | `<% // note /* note */ %>` | No (stays in the servlet source) |

## 13.6 Directives <%@ ... %>

A directive gives the container an **instruction at translation time** about how to translate the page. Three types:

### (a) page directive

| Attribute | Job | Example |
|---|---|---|
| **import** | Import Java packages | `import="java.util.*,java.sql.*"` |
| **contentType** | Response MIME type + charset | `contentType="text/html;charset=UTF-8"` |
| **session** | Enable/disable session (default true) | `session="false"` |
| **errorPage / isErrorPage** | Which page to go to on exception / this is the error page | `errorPage="error.jsp"` |
| **buffer / autoFlush** | Output buffer size / auto flush | `buffer="16kb"` |
| **isELIgnored** | Ignore EL | `isELIgnored="false"` |
| **language / extends / pageEncoding / isThreadSafe** | Language (java), parent class, source encoding, thread model | `pageEncoding="UTF-8"` |

### (b) include directive

JSP

```
<%@ include file="header.jsp" %>      <%-- content is copied in at translation time --%>
```

### (c) taglib directive

```jsp
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>      <%-- for JSTL tags --%>
```

## 13.7 Action tags

| Tag | Job |
|---|---|
| `<jsp:include page="a.jsp" />` | Include the **output** of another resource at runtime |
| `<jsp:forward page="b.jsp" />` | Hand control to another resource (the URL stays the same) |
| `<jsp:param name="k" value="v" />` | Send a parameter along with include/forward |
| `<jsp:useBean> / setProperty / getProperty` | Create/fill/read a JavaBean (Chapter 3) |
| `<jsp:plugin>` | Embed an applet/plugin (old) |

### include directive vs include action

| Point | <%@ include file="" %> | <jsp:include page="" /> |
|---|---|---|
| **When** | **Translation time** | **Request (run) time** |
| **What is included** | Source code is copied | Output (response) is included |
| **Result** | One servlet is produced | Separate servlet, called at runtime |
| **Speed** | Slightly faster | Slightly slower, but flexible |
| **If included file changes** | Parent must be recompiled | Reflected immediately |
| **Use** | Static: header/footer | Dynamic content |

## 13.8 Complete example: all elements together

```jsp
<%@ page import="java.util.*" contentType="text/html;charset=UTF-8" %>
<%!
    int visits = 0;
    String greet(String n) { return "Hello " + n; }
%>
<html><body>
<%
    visits++;
    String name = request.getParameter("n");
    if (name == null) name = "Guest";
%>
<h2><%= greet(name) %></h2>
<p>This page has been viewed <%= visits %> times.</p>
<ul>
<% for (int i = 1; i <= 3; i++) { %>
    <li>Item <%= i %></li>
<% } %>
</ul>
</body></html>
```

### After translation (simplified)

Java

```
public class demo_jsp extends HttpJspBase {
    int visits = 0;                                          // <%! %>  -> class level
    String greet(String n) { return "Hello " + n; }          // <%! %>  -> class level

    public void _jspService(HttpServletRequest request, HttpServletResponse response) ... {
        visits++;                                            // <% %>   -> inside the method
        String name = request.getParameter("n");
        if (name == null) name = "Guest";
        out.print( greet(name) );                            // <%= %>  -> out.print()
        out.print( visits );
        for (int i = 1; i <= 3; i++) { out.write("<li>Item "); out.print(i); out.write("</li>"); }
    }
}
```

**TIP**

**Best practice:** fewer scriptlets, more **EL + JSTL**. The loop above in JSTL: `<c:forEach begin="1" end="3" var="i"><li>Item ${i}</li></c:forEach>`

# 14 Revision, Q&A, Cheat Sheet

Read this right before the exam

## 14.1 One-page Cheat Sheet

| Topic | One line to remember |
|---|---|
| **MVC** | Model = data+logic (Bean/DAO), View = JSP, Controller = Servlet. Request -> Controller -> Model -> Controller -> View -> Response. |
| **Architecture** | Client -> Web tier (Servlet/JSP, Tomcat) -> Business tier (EJB) -> Data tier (DB). Tomcat = web container only. |
| **JavaBean** | public class + no-arg constructor + private fields + getters/setters + Serializable. |
| **EJB** | Server-side business component in an EJB container: transaction, security, pooling for free. Types: Session, MDB (Entity -> JPA). |
| **Session** | HTTP is stateless; HttpSession + JSESSIONID cookie; `getSession(false)` may return null; `invalidate()` for logout. |
| **Session Bean** | Stateless (pool, no state), Stateful (1 client = 1 bean), Singleton (1 per app). Not the same as HttpSession. |
| **Servlet vs JSP session** | Same HttpSession object. In JSP it is the implicit `session`, and session="true" by default. |
| **Servlet lifecycle** | init() once -> service() every request -> destroy() once. One instance, many threads. |
| **JSP lifecycle** | Translate -> Compile -> Load -> Instantiate -> jspInit() -> _jspService() -> jspDestroy(). |
| **JSP vs Servlet** | Servlet = HTML in Java (controller). JSP = Java in HTML (view). A JSP finally becomes a servlet. |
| **Speed** | Same at runtime; JSP is slow on the first request (translate+compile). |
| **9 objects** | request, response, out, session, application, config, pageContext, page, exception. |
| **Elements** | <% %> scriptlet, <%= %> expression, <%! %> declaration, <%@ %> directive, `<jsp:*>` action. |

## 14.2 Interview / Exam Questions (short answers)

**Q1. Why do we use MVC?**

Separation of concerns: keeping UI, logic and control apart makes maintenance, reuse and testing easier.

**Q2. What are the 3 parts of MVC in Servlet/JSP?**

Model = JavaBean/DAO, View = JSP, Controller = Servlet.

**Q3. Rules of a JavaBean?**

Public class, public no-arg constructor, private properties, public getters/setters, Serializable.

**Q4. What does the EJB container provide?**

Transactions, security, pooling, concurrency, lifecycle, DI, remote access.

**Q5. Difference between Stateless and Stateful bean?**

Stateless keeps no client state and is shared from a pool; Stateful has a separate instance per client and remembers state.

**Q6. Are HttpSession and Session Bean the same?**

No. HttpSession is a web-tier object for user data; a Session Bean is an EJB-tier business component.

**Q7. getSession() vs getSession(false)?**

getSession() creates a new session if none exists; getSession(false) returns null.

**Q8. Are Servlet and JSP sessions different?**

No, both are HttpSession. In JSP it is available as the implicit object session.

**Q9. State the JSP life cycle.**

Translation, compilation, loading, instantiation, jspInit(), _jspService(), jspDestroy().

**Q10. Why can't _jspService() be overridden?**

Because the container generates it from the JSP body; that is what the underscore signals.

**Q11. Why are servlet instance variables unsafe?**

A servlet has one object shared by many threads, so race conditions can occur.

**Q12. include directive vs action?**
The directive copies source at translation time; the action includes output at runtime.

**Q13. Why no semicolon in an expression?**
Because it goes inside out.print(expr); a semicolon causes a syntax error.

**Q14. Which JSP object is for exception handling?**
exception, available only in a page with isErrorPage="true".

**Q15. forward vs sendRedirect?**
forward is server-side (same URL, same request, fast); sendRedirect is client-side (new request, URL changes).

## 14.3 Common mistakes

- Putting DB code/logic in a JSP (breaking MVC).
- Keeping user data in servlet instance variables.
- Adding a semicolon in `<%= x; %>`.
- Using `request` or `session` inside `<%! %>`.
- Creating an EJB with `new`.
- Using the result of `getSession(false)` without a null check.
- Forgetting `session.invalidate()` on logout.

> **TIP**
> **All the best!** First practise drawing the MVC and lifecycle diagrams yourself - in an exam a diagram + 2 lines of explanation earns full marks.